

PROGRAMA INTEGRAL DE FORMACIÓN EN PYTHON

CURSO 3: CREACIÓN Y DESARROLLO DE AGENTES DE IA (NIVEL 3
(AVANZADO))

Módulo 01: Fundamentos de IA Generativa y LLMs

«El Cerebro Probabilístico y el Molde de Salida»

*Integración de Modelos de Lenguaje Grande (LLMs): arquitectura Transformer, APIs modernas
(Gemini, OpenAI, Ollama), Prompt Engineering avanzado y salidas estructuradas JSON con
Pydantic.*

Instructor: William Rodríguez (Wisrovi)

Rol: AI Solutions Architect & Principal Software Engineer

Nivel del Curso: Avanzado | **Python:** 3.10+ | **Licencia:** MIT

Acerca del Autor y Mentor



William Rodríguez (Wisrovi)

AI Solutions Architect & Principal Software Engineer • Badajoz, España

Ingeniero y arquitecto de software especializado en Inteligencia Artificial Generativa, sistemas multi-agente, Visión por Computador e infraestructuras MLOps de alta disponibilidad. Creador y mantenedor de la suite de software libre **wisrovi SUITE** en PyPI con más de 26 bibliotecas enfocadas en orquestación de pipelines, caching distribuido y optimización de bases de datos.



GitHub: github.com/wisrovi



LinkedIn: [in/wisrovi-rodriguez](https://in.linkedin.com/in/wisrovi-rodriguez)



DockerHub: hub.docker.com/u/wisrovi



Website: wisrovi.dev

Metodología de Aprendizaje: La Regla de la Bicicleta 🚲

En este programa no creemos en el aprendizaje pasivo. Programar no se aprende memorizando manuales o mirando videos en segundo plano mientras tomas café; se aprende **escribiendo código**, enfrentando errores de sintaxis, depurando variables y viendo cómo responde el intérprete en tiempo real.



El Compromiso Activo del Estudiante

Abre Visual Studio Code en cada sesión. Escribe cada ejemplo con tus propias manos. Cambia los números, rompe el código deliberadamente para ver el mensaje de error de Python, y luego arréglalo. Ese proceso de experimentación es el que construye sinapsis duraderas.

Estructura Pedagógica de Cada Guía

Cada documento de esta serie está diseñado rigurosamente bajo el estándar de ingeniería de software profesional:

- **Fundamento Conceptual y Metáfora Intuitiva:** Anclaje visual del concepto abstracto a la realidad.
- **Diagrama de Flujo y Arquitectura Mental:** Representación visual de cómo fluye la información.
- **Código de Nivel Profesional:** Sintaxis limpia, comentarios línea a línea y tipado estático (PEP 484).
- **Patrones y Gotchas:** Errores comunes que cometen los desarrolladores y cómo evitarlos.

Índice General de la Sesión

Sección	Contenido y Enfoque Temático	Página
Capítulo 1	Fundamentos y Metáfora Conceptual: Arquitectura de Modelos de Lenguaje Grande (LLMs)	Pág. 4
Capítulo 2	Diagrama de Flujo y Arquitectura: Pipeline de Inferencia y Validación Estructurada	Pág. 5
Capítulo 3	Implementación Práctica en Python: Extractor de Entidades con Pydantic y LLM	Pág. 6
Capítulo 4	Patrones Avanzados, Gotchas y Debugging: Gotchas en Integración con LLMs	Pág. 7
Cierre	Conclusiones, Notas del Mentor y Agradecimiento	Pág. 8
Anexos	Bibliografía Oficial y Enlaces de Profundización	Pág. 9

Objetivos de Aprendizaje (Competencias Clave)



Competencia Conceptual

Comprender la naturaleza probabilística de los LLMs, el cálculo de tokens, la ventana de contexto y el control determinista de temperatura.



Competencia Práctica

Construir clientes robustos de IA en Python con validación estricta de esquemas de respuesta tipados.

Requisitos Previos y Entorno Recomendado

Para aprovechar al máximo esta sesión se recomienda tener instalado Python 3.10 o superior, Visual Studio Code con la extensión oficial de Python configurada, y haber completado las lecturas y ejercicios de las sesiones anteriores de la ruta formativa.

1. Arquitectura de Modelos de Lenguaje Grande (LLMs)

Los LLMs no 'piensan' como los humanos; son gigantescas redes neuronales que predicen la siguiente palabra más probable dado un contexto.

☀ **Metáfora Central: El Cerebro Probabilístico y el Molde de Salida**

Un LLM es como un erudito que ha leído toda la biblioteca de Alejandría: si le haces una pregunta abierta responderá con fluidez literaria, pero si le colocas un molde rígido (un esquema JSON con Pydantic), vertirá su conocimiento exclusivamente dentro de la forma exacta que necesitas.

Principios Teóricos y Modelo Mental

Tokens y Contexto: Los textos se tokenizan en fragmentos sub-palabra; la ventana de contexto limita cuántos tokens puede procesar simultáneamente.

Parámetros Clave: Temperatura (0.0 para respuestas deterministas y código; 0.7+ para creatividad), Top-P y penalización de repetición.

⚡ **Regla de Oro en Python**

En entornos de producción nunca uses texto libre del LLM; fuerza siempre salidas tipadas estructuradas validadas con Pydantic.

2. Pipeline de Inferencia y Validación Estructurada

Flujo desde la construcción del System Prompt hasta la validación del objeto de salida.



Desglose Paso a Paso del Diagrama

Fase del Flujo	Acción del Intérprete	Estado en Memoria
1. Inicialización	Construcción del System Prompt con instrucciones de rol y Few-Shot examples.	Tokenización del prompt
2. Evaluación	Envío a la API del modelo (Gemini / OpenAI / Ollama) con schema JSON.	Inferencia en la GPU
3. Transformación	El modelo genera un payload JSON estricto cumpliendo la especificación.	Payload JSON crudo
4. Retorno / Salida	Pydantic parsea y valida los tipos de datos en un objeto Python listo.	Instancia BaseModel validada

Visualización Mental

Trata al LLM como un microservicio no determinista: coloca siempre una capa de validación antes de entregar los datos a tu backend.

3. Extractor de Entidades con Pydantic y LLM

Esquema tipado para forzar respuestas deterministas en Python:

```
main.py (Python 3.10+) UTF-8 • PEP 8 Compliant

from pydantic import BaseModel, Field
from typing import List

# Esquema de validación estricta
class AnalisisSentimiento(BaseModel):
    sentimiento: str = Field(description="POSITIVO, NEGATIVO o NEUTRO")
    puntuacion_confianza: float = Field(ge=0.0, le=1.0)
    temas_clave: List[str] = Field(default_factory=list)
    resumen_ejecutivo: str

# Simulación de respuesta parseada por el motor
payload_llm = '''{
  "sentimiento": "POSITIVO",
  "puntuacion_confianza": 0.96,
  "temas_clave": ["soporte rápido", "calidad software", "precio justo"],
  "resumen_ejecutivo": "El cliente expresa gran satisfacción con la atención recibida."
}'''

analisis = AnalisisSentimiento.model_validate_json(payload_llm)
print(f"Sentimiento: {analisis.sentimiento} ({analisis.puntuacion_confianza*100:.1f}%)")
print(f"Temas: {'', '.join(analisis.temas_clave)}")
```

Análisis del Código Fuente

Uso de Pydantic V2 para validación robusta con límites de rango numérico (ge, le) y serialización JSON directa.

4. Gotchas en Integración con LLMs

Errores frecuentes al conectar modelos de IA generativa a sistemas de software:

⚠ Gotcha Frecuente (Trampa de Principiante)

Confiar ciegamente en que el LLM siempre responderá JSON válido sin capturar excepciones de parseo o alucinaciones.

Patrón Recomendado vs Antipatrón

❌ Antipatrón / Mal Código

```
data = json.loads(llm_response) # Fallará si el
LLM incluye texto extra
```

✅ Patrón Pythonic / Correcto

```
try:
    data =
    Model.model_validate_json(llm_response)
except ValidationError as e:
    # Estrategia de reintento / corrección
```

🛡 Consejo de Resiliencia en Producción

Utiliza Temperature=0.0 para extracción de datos, clasificación y generación de código reproducible.

5. Resumen Ejecutivo y Conclusiones

Comprendes los fundamentos de la IA generativa y sabes conectar modelos LLM con validación de esquemas tipados.



Logro Alcanzado en esta Clase

Capacidad para construir tuberías de datos asistidas por IA que no fallen en producción.

Notas del Instructor y Siguientes Pasos

En el siguiente módulo aprenderemos Tool Calling (Function Calling), Memoria y RAG con bases de datos vectoriales.



Mensaje de Agradecimiento

Muchas gracias por tu entusiasmo, disciplina y dedicación al participar en este programa formativo. La programación es un superpoder que transforma vidas cuando se ejerce con constancia y curiosidad. ¡Nos vemos en la próxima sesión para seguir construyendo juntos!

«El código más elegante es aquel que cualquiera puede entender y nadie necesita reescribir.»

6. Fuentes Bibliográficas y Recursos de Estudio

Para profundizar y consolidar los conocimientos adquiridos en esta clase, consulta las siguientes referencias:

Recurso / Fuente	Descripción Temática	Enlace Oficial
Documentación Oficial de Python	Referencia canónica del lenguaje y librería estándar	docs.python.org/3/
PEP 8 - Style Guide for Python	Guía oficial de estilo, formato e indentación	peps.python.org
Real Python Tutorials	Artículos técnicos y patrones de desarrollo moderno	realpython.com
Python Type Checking (PEP 484)	Anotaciones de tipo y análisis estático en Python	docs.python.org/typing
Suite Open Source wisrovi	Paquetes Python para orquestación y alto rendimiento	github.com/wisrovi

Canales de Consulta y Comunidad

Si encuentras dificultades al replicar el código o tienes dudas sobre la arquitectura de los ejercicios, no dudes en abrir un Issue en el repositorio oficial del curso en GitHub o participar activamente en nuestras sesiones grupales de mentoría.



Desafío de Autoestudio Recomendado

Crea un script que consulte la API de Gemini u Ollama para resumir un artículo largo forzando salida en JSON con Pydantic.